

Runwise Developer Guide

Stack

Prerequisites

- Node.js 22 or later
 - npm
-

Setup

```
git clone https://github.com/alanwaddington/runwise.git
cd runwise
npm install
npm run dev
```

Open <http://localhost:5173>.

Project Structure

```
src/
% % % app.css           # Global styles + Tailwind theme tokens
% % % app.html          # SvelteKit HTML shell
% % % lib/
% % % components/       # Shared UI components
% % %   HeroSection.svelte
% % %   HeroSection.test.ts
% % %   InputField.svelte
% % %   InputField.test.ts
% % %   ResultDisplay.svelte
% % %   ResultDisplay.test.ts
% % %   SiteNav.svelte
% % %   SiteNav.test.ts
% % %   ToolCard.svelte
% % %   ToolCard.test.ts
% % %   ToolLayout.svelte
% % %   ToolLayout.test.ts
% % %   utils/          # Pure utility modules (no Svelte dependency)
% % %     pace.ts        # Pace/speed conversion functions
% % %     pace.test.ts
% % %     race-predictor.ts  # Riegel formula, time parsing/formatting, prediction
table
% % %   race-predictor.test.ts
% % % routes/
% % %   +layout.svelte  # Root layout – header + main wrapper
% % %   +page.svelte    # Home page – HeroSection + ToolCard grid
% % %   +error.svelte   # Error page
% % %   pace/
% % %   race-predictor/
% % %   training-paces/
% % %   hr-zones/
% % %   vo2max/
% % %   parkrun/
```

Design System

Tokens (`src/app.css`)

Dark mode is applied automatically via prefers-color-scheme. Always use design tokens (text-ink, bg-bg, border-ink/10) rather than hardcoded Tailwind colours.

Components

Adding a New Tool

```
Create `src/routes/<tool-name>/+page.svelte`
Import `ToolLayout` and pass `title` and `description` props
Add the tool to the `tools` array in `src/routes/+page.svelte`
Add the tool link to `SiteNav.svelte`
Add tests alongside the page
```

Testing

Unit and component tests (Vitest)

```
npm run test          # Run all tests once
npm run test -- --watch # Watch mode
```

Tests live alongside their source files (*.test.ts). Follow the existing pattern:

```
import { describe, it, expect, afterEach } from 'vitest';
import { render, cleanup, screen } from '@testing-library/svelte';
import MyComponent from './MyComponent.svelte';

afterEach(() => { cleanup(); });

describe('MyComponent', () => {
  it('renders correctly', () => {
    render(MyComponent, { props: { ... } });
    expect(screen.getByRole('...')).toBeInTheDocument();
  });
});
```

E2E tests (Playwright)

```
npx playwright install chromium # First time only
npm run test:e2e                # Run E2E tests against the production build
```

E2E tests live in the e2e/ directory. They run against the production preview server (npm run build && npm run preview on port 4173). Configuration is in playwright.config.ts.

Code Quality

```
npm run check      # TypeScript / svelte-check
npm run lint       # ESLint
npm run format     # Prettier (auto-fix)
npm run test:e2e   # Playwright E2E tests (requires built app)
```

Workflow

This project follows an issue-driven workflow:

```
/analyse <issue>    !' requirements + acceptance criteria
/design <issue>     !' architecture + work breakdown
/develop <issue>    !' TDD implementation
/verify <PR>        !' runtime verification
/pr-reviewer <PR>   !' acceptance criteria audit
/merge <PR>         !' merge + close issues
```